

ui.context_menu 全面详解

ui.context_menu 是 NiceGUI 专门用于实现**右键上下文菜单**的组件，基于 Quasar 底层组件封装，支持右键点击任意元素或页面空白处触发，可嵌套菜单项、子菜单和分隔线，核心用于提供快捷操作入口（如表格行右键删除、文本右键复制、页面右键自定义功能等）。其核心特性包括自动贴合触发元素、支持多级嵌套、点击外部自动关闭、样式自定义等，是替代传统 `ui.menu(on_context_menu=True)` 的更简洁、语义化的解决方案。

一、核心概念与基础特性

1. 本质与用途

- 本质：语义化的右键菜单组件，封装了「右键触发」的核心逻辑，无需手动配置 `on_context_menu` 参数，直接绑定触发元素即可实现右键唤醒。
- 核心用途：为界面元素或页面提供上下文快捷操作，节省界面空间，提升操作效率，常见场景包括：
 - 表格 / 列表行：右键删除、编辑、复制数据；
 - 文本 / 图片：右键复制、下载、分享；
 - 页面空白处：右键刷新、返回顶部、切换主题；
 - 文件管理器：右键新建、重命名、删除文件。
- 关键机制：
 - 触发方式：默认右键点击绑定的元素唤醒，无需额外配置；
 - 显示位置：自动贴合鼠标点击位置，确保菜单完全可见（页面边缘会自动调整）；
 - 关闭逻辑：点击菜单项、点击菜单外部、触发其他上下文菜单时自动关闭；
 - 嵌套支持：可嵌套 `ui.submenu` 实现多级子菜单，支持 `ui.menu_divider` 分隔选项。

2. 基础结构

ui.context_menu 需绑定到触发元素（如按钮、表格、页面），内部嵌套 `ui.menu_item`（菜单项）、`ui.submenu`（子菜单）或 `ui.menu_divider`（分隔线），基础使用示例如下：

```
from nicegui import ui

# 1. 绑定到具体元素（按钮右键菜单）
with ui.button('右键点击我', classes='mt-4') as trigger:
    with ui.context_menu(trigger=trigger):
        ui.menu_item('新建文件', on_click=lambda: ui.notify('新建文件'))
        ui.menu_item('打开文件', on_click=lambda: ui.notify('打开文件'))
        ui.menu_divider()
        with ui.submenu('导出'):
            ui.menu_item('导出为 PDF', on_click=lambda: ui.notify('导出 PDF'))
            ui.menu_item('导出为 Excel', on_click=lambda: ui.notify('导出 Excel'))

# 2. 绑定到页面（空白处右键菜单）
with ui.context_menu(trigger=ui.page):
    ui.menu_item('刷新页面', on_click=lambda: ui.notify('页面已刷新'))
    ui.menu_item('返回顶部', on_click=lambda: ui.page.scroll_to_top())
    ui.menu_item('切换主题', on_click=lambda: ui.dark_mode.toggle())
```

`ui.run()`

二、初始化配置项

初始化 `ui.context_menu()` 时可通过参数配置触发元素、显示行为、回调等核心属性，参数说明如下（基于官方文档核心配置，补充实战常用参数）：

参数名	类型	说明
trigger	Element Page	菜单触发元素（必填），可绑定到任意 NiceGUI 元素（如 <code>ui.button</code> 、 <code>ui.table</code> ）或 <code>ui.page</code> （页面空白处）
value	bool	初始显示状态（默认 <code>False</code> ，即隐藏；设为 <code>True</code> 时初始显示，通常无需手动设置）
on_value_change	Callable	菜单显示 / 隐藏状态变化时触发的回调函数（接收 <code>ValueChangeEventArguments</code> 参数， <code>e.value</code> 为当前状态）
close_on_click	bool	点击菜单项后是否关闭菜单（默认 <code>True</code> ，点击后关闭；设为 <code>False</code> 时保持打开，适用于多选逻辑）
offset	Tuple[int, int]	菜单显示位置偏移量（单位：像素），格式为 <code>(x_offset, y_offset)</code> （默认无偏移，自动贴合鼠标位置）
max_height	str int	菜单最大高度（支持像素值如 <code>300</code> 、百分比如 <code>'80%'</code> ，超出部分滚动显示）
prevent_default	bool	是否阻止浏览器默认右键菜单（默认 <code>True</code> ，避免与自定义菜单冲突；设为 <code>False</code> 时会同时显示浏览器默认菜单）

配置示例

```
from nicegui import ui

# 自定义偏移、最大高度、点击不关闭菜单的上下文菜单
def on_menu_toggle(e):
    ui.notify(f'上下文菜单已{"显示" if e.value else "隐藏"}')

with ui.card('右键点击卡片', classes='w-64 h-32 flex items-center justify-center mt-4') as trigger:
    with ui.context_menu(
        trigger=trigger,
        close_on_click=False, # 点击菜单项不关闭菜单
        offset=(10, 10), # 菜单偏移 (x+10px, y+10px)
        max_height=200, # 最大高度 200px, 超出滚动
        prevent_default=True, # 阻止浏览器默认右键菜单
        on_value_change=on_menu_toggle
    ):
        ui.menu_item('选项1', on_click=lambda: ui.notify('选择选项1'))
        ui.menu_item('选项2', on_click=lambda: ui.notify('选择选项2'))
```

```
# 超出最大高度的菜单项会自动滚动
for i in range(3, 11):
    ui.menu_item(f'选项{i}', on_click=lambda i=i: ui.notify(f'选择选项{i}'))

ui.run()
```

三、核心属性

ui.context_menu 继承 NiceGUI 基础元素的通用属性，支持样式配置、状态绑定等，关键属性如下（含可动态修改的属性）：

属性名	类型	说明
classes	str	元素的 CSS 类名（支持 Tailwind、Quasar 类，如 <code>w-48</code> 固定宽度、 <code>bg-gray-50</code> 背景色、 <code>shadow-1g</code> 阴影）
props	str	Quasar 组件属性（用于扩展功能，如 <code>color=primary</code> 设置颜色、 <code>rounded-1g</code> 圆角、 <code>border</code> 边框）
style	str	内联 CSS 样式（如 <code>font-size: 14px;</code> 调整字体大小、 <code>padding: 4px 0;</code> 调整内边距）
value	BindableProperty	菜单显示状态（布尔值， <code>True</code> 显示、 <code>False</code> 隐藏，支持双向绑定）
visible	BindableProperty	元素可见性（布尔值，支持动态绑定，与 <code>value</code> 区别： <code>visible</code> 控制元素是否存在， <code>value</code> 控制菜单是否展开）
enabled	BindableProperty	菜单是否启用（布尔值， <code>False</code> 时无法通过右键触发显示）
trigger	Element Page（可设置）	动态修改触发元素（如 <code>context_menu.trigger = new_card</code> ）
close_on_click	bool（可设置）	动态修改点击菜单项后是否关闭菜单（如 <code>context_menu.close_on_click = True</code> ）
offset	Tuple [int, int]（可设置）	动态修改偏移量（如 <code>context_menu.offset = (20, 20)</code> ）
max_height	str int（可设置）	动态修改最大高度（如 <code>context_menu.max_height = 300</code> ）
prevent_default	bool（可设置）	动态切换是否阻止浏览器默认菜单（如 <code>context_menu.prevent_default = False</code> ）
html_id	str	HTML DOM 中的元素 ID（版本 2.16.0 新增，用于精准定位）
is_deleted	bool	元素是否已被删除（只读属性）
parent_slot	Slot None	父容器的插槽（可手动设置，用于复杂布局嵌套）

属性使用示例

```
from nicegui import ui

# 自定义样式的上下文菜单：固定宽度、圆角、阴影、灰色背景
with ui.button('自定义样式菜单', classes='mt-4') as trigger:
    context_menu = ui.context_menu(trigger=trigger).props('rounded-lg shadow-lg color=blue border').classes('w-56 bg-gray-50')
    ui.menu_item('新建项目', icon='add').props('hover:bg-blue-50') # hover 效果
    ui.menu_item('打开项目', icon='folder_open').props('hover:bg-blue-50')
    ui.menu_divider(props='color=blue-200')
    ui.menu_item('设置', icon='settings').props('hover:bg-blue-50')

# 动态修改菜单属性
def update_menu_config():
    context_menu.offset = (15, 15) if context_menu.offset == (0, 0) else (0, 0)
    context_menu.prevent_default = not context_menu.prevent_default
    ui.notify(f'偏移: {context_menu.offset}, 阻止默认菜单: {context_menu.prevent_default}')

ui.button('切换菜单配置', on_click=update_menu_config).classes('mt-2')
```

四、核心方法

ui.context_menu 提供丰富的方法用于控制菜单显示 / 隐藏、绑定状态、管理子元素等，常用方法分类如下：

1. 菜单状态控制方法

方法名	作用	示例
show()	手动显示菜单（通常无需手动调用，右键自动触发）	<code>context_menu.show()</code>
hide()	手动隐藏菜单（点击外部或菜单项后自动调用）	<code>context_menu.hide()</code>
toggle()	切换菜单显示 / 隐藏状态	<code>context_menu.toggle()</code>
set_value(value: bool)	设置菜单显示状态（True 显示，False 隐藏）	<code>context_menu.set_value(True)</code>
enable()	启用菜单（允许通过右键触发显示）	<code>context_menu.enable()</code>
disable()	禁用菜单（禁止通过右键触发显示）	<code>context_menu.disable()</code>
set_enabled(value: bool)	动态设置启用状态	<code>context_menu.set_enabled(False)</code>

2. 绑定方法

支持将菜单显示状态、启用状态、可见性与外部变量绑定，实现状态同步，核心方法如下：

方法名	作用	关键参数
<code>bind_value(target_object, target_name='value')</code>	双向绑定菜单显示状态到目标对象的属性	<code>target_object</code> : 绑定目标对象; <code>target_name</code> : 绑定的属性名 (默认 'value')
<code>bind_value_from(...)</code>	单向绑定 (从目标对象同步到菜单)	同 <code>bind_value</code> , 仅单向同步 (目标对象属性变化触发菜单显示状态变化)
<code>bind_value_to(...)</code>	单向绑定 (从菜单同步到目标对象)	同 <code>bind_value</code> , 仅单向同步 (菜单显示状态变化触发目标对象属性变化)
<code>bind_enabled(...)</code>	双向绑定启用状态到目标对象的属性	同 <code>bind_value</code> , 绑定的属性为启用状态 (默认 'enabled')
<code>bind_visibility(...)</code>	双向绑定可见性到目标对象的属性	<code>value</code> : 可选, 指定目标值匹配时才显示菜单元素

3. 子元素管理方法

方法名	作用	参数说明
<code>clear()</code>	删除菜单内所有子元素 (菜单项、子菜单、分隔线)	无参数
<code>remove(element)</code>	删除指定子元素	<code>element</code> : 子元素实例 (<code>ui.menu_item</code> / <code>ui.submenu</code> / <code>ui.menu_divider</code> 对象) 或其 ID
<code>add_item(label: str, on_click: Callable = None, **kwargs)</code>	快速添加菜单项	<code>label</code> : 菜单项文本; <code>on_click</code> : 点击回调; <code>**kwargs</code> : 其他参数 (如 <code>icon</code> 、 <code>props</code>)
<code>add_divider()</code>	快速添加菜单分隔线	无参数
<code>delete()</code>	删除整个上下文菜单元素及所有子元素	无参数

4. 其他常用方法

方法名	作用	示例
tooltip(text)	为触发元素添加鼠标悬浮提示（菜单本身无 tooltip，需绑定到触发元素）	<code>trigger.tooltip('右键打开菜单')</code>
update()	强制更新客户端的菜单状态（如动态添加子元素后刷新）	<code>context_menu.update()</code>
add_resource(path)	为菜单添加资源文件（如自定义 CSS、JS）	<code>context_menu.add_resource('./static')</code>
mark(*markers)	为元素添加标记（用于测试或元素查询）	<code>context_menu.mark('table-context-menu', 'delete')</code>

方法使用示例

```
from nicegui import ui

# 绑定菜单显示状态到外部变量
class MenuState:
    is_open = False
    is_enabled = True

state = MenuState()

with ui.button('状态绑定菜单', classes='mt-4') as trigger:
    context_menu = ui.context_menu(trigger=trigger)
    # 双向绑定显示状态和启用状态
    context_menu.bind_value(state, 'is_open')
    context_menu.bind_enabled(state, 'is_enabled')
    # 初始菜单项
    context_menu.add_item('选项1', on_click=lambda: ui.notify('选择选项1'))
    context_menu.add_item('选项2', on_click=lambda: ui.notify('选择选项2'))
    context_menu.add_divider()

# 显示绑定的状态
ui.label('菜单状态: ').bind_text_from(state, 'is_open', lambda s: '打开' if s else '关闭').classes('mt-2')
ui.label('启用状态: ').bind_text_from(state, 'is_enabled', lambda e: '启用' if e else '禁用').classes('mt-1')

# 程序控制菜单
ui.row(
    ui.button('显示菜单', on_click=context_menu.show),
    ui.button('隐藏菜单', on_click=context_menu.hide),
    ui.button('切换启用', on_click=lambda: setattr(state, 'is_enabled', not state.is_enabled)),
)
```

```
        ui.button('添加菜单项', on_click=lambda: context_menu.add_item('动态选项', on_click=lambda:
        ui.notify('动态选项被点击'))))
    ).classes('mt-2')

    ui.run()
```

五、子元素详解（与 ui.menu 共用）

ui.context_menu 的子元素与 `ui.menu` 完全一致，核心包括 `ui.menu_item`（菜单项）、`ui.submenu`（子菜单）、`ui.menu_divider`（分隔线），具体配置如下：

1. ui.menu_item（菜单项）

核心参数

参数名	类型	说明
label	str	菜单项文本（必填）
on_click	Callable	点击回调（可选，无回调时为纯展示项）
icon	str	左侧图标（支持 Quasar 图标库名称，如 'delete'、'edit'）
props	str	样式属性（如 <code>color=red</code> 红色文本、 <code>disabled</code> 禁用、 <code>active</code> 激活状态）
classes	str	CSS 类名（如 <code>text-bold</code> 加粗、 <code>py-2</code> 调整内边距）

使用示例

```
# 带图标、禁用、激活状态的菜单项
ui.menu_item('编辑', icon='edit', on_click=lambda: ui.notify('编辑数据'))
ui.menu_item('删除', icon='delete', on_click=lambda: ui.notify('删除数据')).props('color=red')
ui.menu_item('复制', icon='content_copy', on_click=lambda: ui.notify('复制数据')).props('active')
ui.menu_item('分享', icon='share', on_click=lambda: ui.notify('分享数据')).props('disabled')
```

2. ui.submenu（子菜单）

核心参数

参数名	类型	说明
label	str	子菜单触发文本（必填）
icon	str	左侧图标（支持 Quasar 图标库名称）
props	str	样式属性（如 <code>color=blue</code> 、 <code>rounded</code> ）

使用示例

```
# 多级子菜单嵌套
with ui.submenu('导出', icon='file_export'):
    ui.menu_item('导出为 PDF', on_click=lambda: ui.notify('导出 PDF'))
    ui.menu_item('导出为 Excel', on_click=lambda: ui.notify('导出 Excel'))
    with ui.submenu('更多格式'): # 二级子菜单
        ui.menu_item('导出为 CSV', on_click=lambda: ui.notify('导出 CSV'))
        ui.menu_item('导出为 TXT', on_click=lambda: ui.notify('导出 TXT'))
```

3. ui.menu_divider (分隔线)

核心参数

参数名	类型	说明
props	str	样式属性 (如 color=gray-300 调整分隔线颜色)
classes	str	CSS 类名 (如 my-1 调整上下边距)

使用示例

```
ui.menu_item('选项1')
ui.menu_divider(props='color=blue-200').classes('my-1') # 蓝色分隔线，上下边距 1px
ui.menu_item('选项2')
```

六、 事件处理

1. 核心事件： 菜单显示 / 隐藏状态变化事件

通过 on_value_change 绑定状态变化回调，实时监听菜单显示 / 隐藏：

```
def on_menu_change(e):
    ui.notify(f'上下文菜单状态: {"显示" if e.value else "隐藏"}')

with ui.button('状态监听菜单', classes='mt-4') as trigger:
    ui.context_menu(trigger=trigger, on_value_change=on_menu_change)
    ui.menu_item('选项1', on_click=lambda: ui.notify('选择选项1'))
```

2. 菜单项点击事件

通过 ui.menu_item 的 on_click 参数或 on() 方法绑定，支持动态修改：


```

with ui.button('点击事件菜单', classes='mt-4') as trigger:
    with ui.context_menu(trigger=trigger):
        # 方式1: 初始化绑定
        ui.menu_item('初始回调', on_click=lambda: ui.notify('初始回调触发'))

        # 方式2: 动态绑定
        item = ui.menu_item('动态回调', icon='event')
        item.on('click', lambda: ui.notify('动态绑定的回调触发'))

```

3. 通用 DOM 事件

为上下文菜单容器绑定鼠标悬浮、离开等事件（较少用，主要用于特殊交互）：

```

with ui.button('通用事件菜单', classes='mt-4') as trigger:
    context_menu = ui.context_menu(trigger=trigger)
    ui.menu_item('选项1')

    # 鼠标进入菜单容器
    context_menu.on('mouseover', lambda: ui.notify('鼠标进入菜单'))
    # 鼠标离开菜单容器
    context_menu.on('mouseout', lambda: ui.notify('鼠标离开菜单'))

```

七、高级用法（实战场景）

1. 表格行右键菜单（核心场景）

为表格每行数据添加右键快捷操作，结合数据索引实现精准操作：

```

from nicegui import ui

# 模拟表格数据
table_data = [
    {'id': 1, 'name': '图书1', 'author': '作者A', 'status': '可借阅'},
    {'id': 2, 'name': '图书2', 'author': '作者B', 'status': '已借出'},
    {'id': 3, 'name': '图书3', 'author': '作者C', 'status': '可借阅'},
]

# 创建表格
columns = [
    {'name': 'id', 'label': 'ID', 'field': 'id'},
    {'name': 'name', 'label': '书名', 'field': 'name'},
    {'name': 'author', 'label': '作者', 'field': 'author'},
    {'name': 'status', 'label': '状态', 'field': 'status'},
]
table = ui.table(columns=columns, rows=table_data).classes('w-full')

# 为每行添加右键菜单
def create_row_context_menu(row_index):
    """为指定行创建右键菜单"""
    def show_menu(e):
        e.preventDefault() # 阻止浏览器默认菜单

```

```

        row = table_data[row_index]

        with ui.context_menu(trigger=ui.page, prevent_default=True) as menu:
            ui.menu_item(f'编辑 《{row["name"]}》', on_click=lambda: ui.notify(f'编辑图书:
{row["name"]}'))
            # 根据状态动态显示菜单项
            if row['status'] == '可借阅':
                ui.menu_item(f'借出 《{row["name"]}》', on_click=lambda: ui.notify(f'借出图书:
{row["name"]}'))
            else:
                ui.menu_item(f'归还 《{row["name"]}》', on_click=lambda: ui.notify(f'归还图书:
{row["name"]}'))
            ui.menu_divider()
            ui.menu_item(f'删除 《{row["name"]}》', on_click=lambda: ui.notify(f'删除图书:
{row["name"]}')).props('color=red')

        menu.show() # 显示菜单

    return show_menu

# 为表格每行绑定右键事件
for i in range(len(table_data)):
    table.rows[i].on('contextmenu', create_row_context_menu(i))

ui.run()

```

2. 动态菜单管理（添加 / 删除 / 清空）

根据业务逻辑动态修改菜单内容，适用于权限控制、动态功能展示等场景：

```

from nicegui import ui

with ui.button('动态上下文菜单', classes='mt-4') as trigger:
    context_menu = ui.context_menu(trigger=trigger)
    # 初始菜单项
    context_menu.add_item('基础功能1', on_click=lambda: ui.notify('基础功能1'))
    context_menu.add_item('基础功能2', on_click=lambda: ui.notify('基础功能2'))
    context_menu.add_divider()

# 动态添加菜单项
def add_item():
    item_count = len([c for c in context_menu.default_slot.children if hasattr(c, 'label')])
    context_menu.add_item(f'动态功能{item_count+1}', on_click=lambda i=item_count+1:
ui.notify(f'动态功能{i}'))

# 清空菜单
def clear_menu():
    context_menu.clear()
    ui.notify('菜单已清空')

# 动态添加子菜单
def add_submenu():

```

```

with context_menu:
    with ui.submenu('动态子菜单', icon='subdirectory_arrow_right'):
        ui.menu_item('子功能1', on_click=lambda: ui.notify('子功能1'))
        ui.menu_item('子功能2', on_click=lambda: ui.notify('子功能2'))

ui.row(
    ui.button('添加菜单项', on_click=add_item),
    ui.button('添加子菜单', on_click=add_submenu),
    ui.button('清空菜单', on_click=clear_menu)
).classes('mt-2')

ui.run()

```

3. 样式深度自定义（品牌化适配）

结合 Tailwind CSS 和 Quasar props 实现个性化样式，适配产品品牌风格：

```

from nicegui import ui

with ui.button('品牌化菜单', classes='bg-purple-600 text-white mt-4') as trigger:
    with ui.context_menu(trigger=trigger).props('rounded-lg shadow-xl color=purple border border-purple-200').classes('w-64 bg-purple-50'):
        # 自定义菜单项样式 (hover 背景、图标颜色)
        ui.menu_item('首页', icon='home').props('color=purple hover:bg-purple-100').classes('py-3 px-4')
        ui.menu_item('我的订单', icon='shopping_cart').props('color=purple hover:bg-purple-100').classes('py-3 px-4')
        ui.menu_divider(props='color=purple-200').classes('my-1')
        # 子菜单自定义样式
        with ui.submenu('账户中心', icon='account_circle', props='color=purple hover:bg-purple-100').classes('py-3 px-4'):
            ui.menu_item('个人资料', props='hover:bg-purple-100')
            ui.menu_item('退出登录', props='color=red hover:bg-red-100')

ui.run()

```

八、注意事项

1. 触发元素约束：trigger 参数必须是已挂载到 DOM 的 NiceGUI 元素或 ui.page，动态创建的触发元素需确保绑定菜单时已存在。
2. 浏览器默认菜单：默认 prevent_default=True 阻止浏览器默认右键菜单，若需保留（如允许用户复制页面文本），可设为 False，但会与自定义菜单同时显示，需谨慎使用。
3. 菜单显示位置：菜单会自动贴合鼠标点击位置，若触发元素在页面边缘（如底部、右侧），菜单会自动调整方向以确保完全显示，无需手动处理。
4. 多级子菜单：支持多级嵌套，但建议不超过 3 级，避免层级过深导致操作繁琐；子菜单默认点击后关闭父菜单，可通过 auto_close=False 调整。
5. 动态更新后刷新：动态添加 / 删除菜单项或修改菜单属性后，建议调用 update() 方法刷新客户端显示，确保状态同步。
6. 版本兼容性：html_id 属性需 NiceGUI 2.16.0+ 版本支持，bind_enabled 方法需 3.0.0+ 版本支持，使用时

需确认版本匹配。

7. 与 `ui.menu` 的区别: `ui.context_menu` 是 `ui.menu(on_context_menu=True)` 的语义化封装, 无需手动配置触发方式, 更适合专门的右键菜单场景; `ui.menu` 更通用, 支持左键点击、右键触发等多种方式。

通过以上配置与方法, `ui.context_menu` 可灵活满足从简单页面右键菜单到复杂表格行快捷操作的各类需求, 是 NiceGUI 中实现上下文交互的核心组件之一。